

计算机体系结构

L06-4 多处理器同步

计算机体系结构

背景知识 (自学) ③

存储层次 ④

单处理器 (指令级并行) ④

多处理器 (线程级并行)

加速器 (数据并行) ③

多处理分类 ②

Cache 一致性 (Coherence) ②

同步 (Synchronization)

内存一致性 (Consistency) ③

互连网络 (Interconnect) ②

处理器之间同步类型

控制访问顺序

设置同步点

互斥

硬件支持

禁用中断

原子指令

多处理同步 (synchronization)

□ Cache Coherence 协议保证处理器读取数据时总能得到最新值

- 让多处理器系统中的cache看起来像单处理器系统
- 协议关注的是单个cache block

□ 然而, coherence协议无法做到:

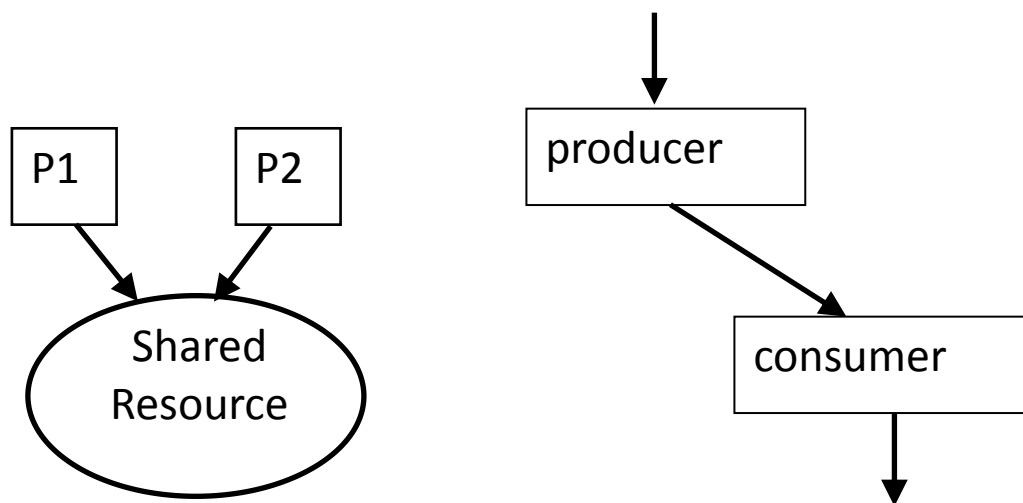
- 指定对多个block的访问交互方式
- 确保处理器按照特定顺序访问共享数据
- 确保一次最多k个处理器访问某共享硬件或者数据
- 强制让多个处理器同时执行某段特殊程序

□ 多处理器的操作需要同步 (*synchronized*)

为何需要同步?

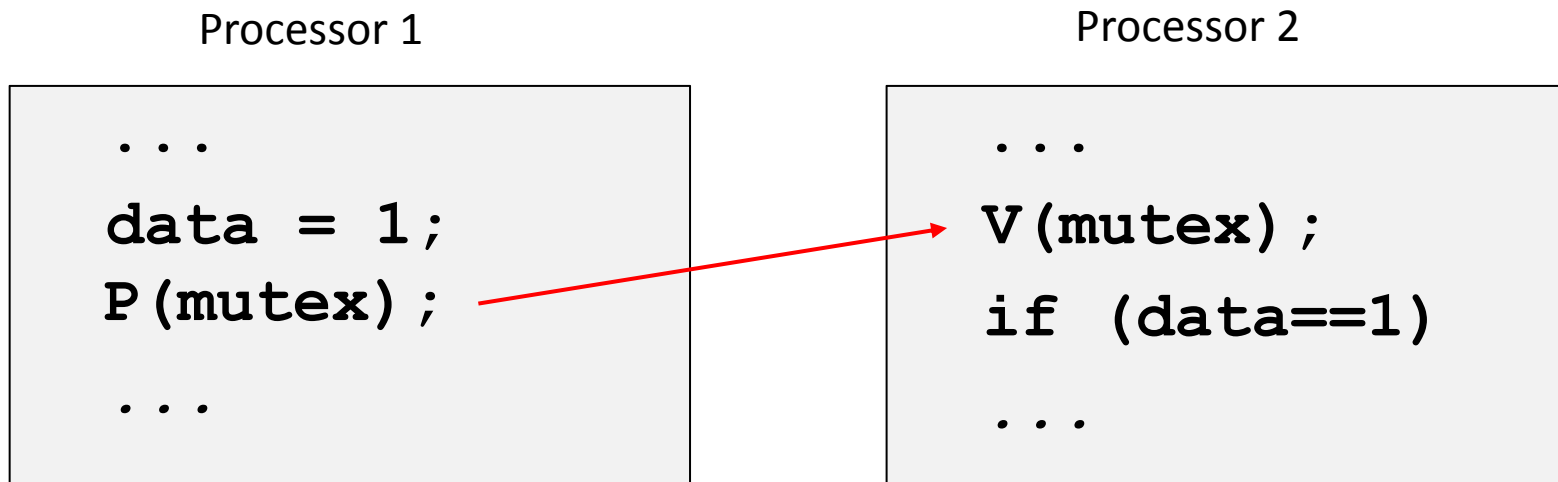
□ 确保不同的线程/进程/处理器可以安全地使用共享数据

- **控制访问顺序**: 消费者需要等到生产者准备好数据
- **控制执行进度**: 所有处理器同时开始执行/到某个节点
- **互斥**: 在某个给定时刻，最多只有一个处理器可以使用硬件或者数据



同步需求（1）：控制访问顺序

- 在threads/processes/processors之间传递数据
- 使用信号量: P(S) and V(S)



如果data 和 mutex 都在本地的私有cache中会如何?

同步需求（2）：控制执行进度

- 在很多threads/processes/processors之间同步操作
- 采用barrier: barrier()

Processor 1

```
...  
data[1] = a;  
barrier(B1);  
sum += data[1];  
...
```



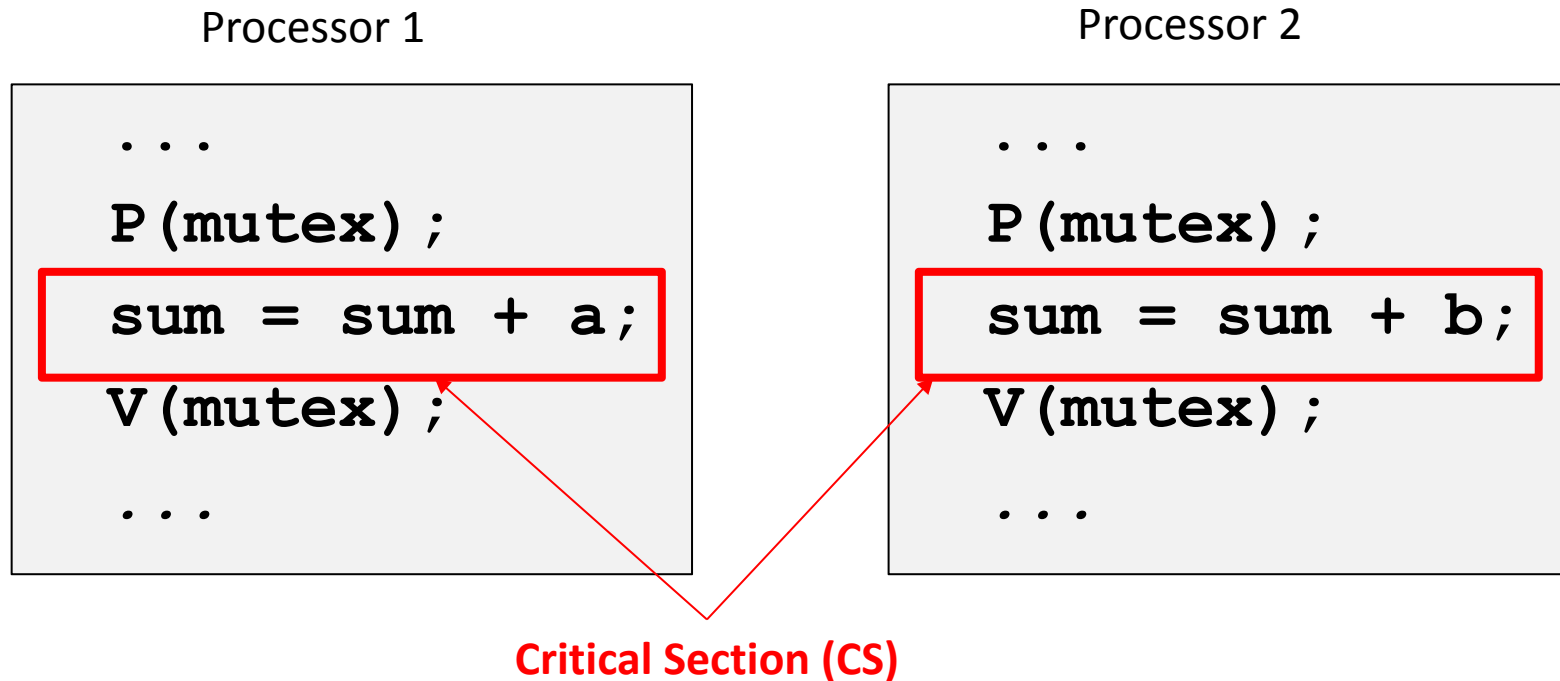
Processor 2

```
...  
data[2] = b;  
barrier(B1);  
sum += data[2];  
...
```



同步需求 (3) : 互斥

- 最多让1个threads/processes/processors访问共享数据
- 使用锁或者信号量: `lock()/unlock()` or `P()/V()`



临界区

□ 一个代码序列，**只有一个线程/处理器**可以在同一时间执行

– 提供互斥功能

- 一个线程/处理器对代码及其访问的数据拥有独占访问权 → 间接保护
- 保证每次最多只有一个线程/处理器可以更新数据

– 为了执行临界区, 一个线程/处理器

- 获取一个保护它的锁/信号量
- 执行代码以操作受保护的数据
- 释放锁/信号量

□ 其效果是同步/排序线程(处理器)对共享数据的访问

多处理器同步的硬件支持

□ 很多系统在硬件层面提供对临界区的支持

□ **单处理器:** 可以禁用中断

- 当前正在运行的代码将不会因为中断而被切换出去
- 系统时钟仍需通过中断进行更新

□ 一个简单的锁的实现:

```
LockAcquire { disable Ints; }
```

```
LockRelease { enable Ints; }
```

□ 该方法的问题:

- 临界区不能太长, 否则系统无法及时响应中断

多处理器同步的硬件支持

□ 特殊原子指令(atomic)

- 原子性 = 不可打断
- 原子指令允许test and modify一个数或者swap两个数的值

□ 基本的原子操作:

- **Atomic exchange**: 交换两个数的值
- **Test-and-set**: 在特定条件下设置内存值
- **Fetch-and-increment**: 读取内存原来的值，增加之后再放回内存
- **Load linked/store conditional**: load linked读取数据并记录内存位置，之后store conditional尝试写该位置，如果在load linked和store conditional之间没有别的指令更新该位置，那么store conditional成功，否则失败

原子交换操作

□ 在一个原子操作中交换寄存器的值和内存的值

- 将R2先置为 1
- 交换R2和锁变量(内存0(R1)，初始值为0)
 - 在一个原子操作中包含一个load和一个store操作
- 交换后的R2决定了是否拿到了锁
 - 如果R2为0，表明交换成功，抢到了锁
- 使用atomic exchange实现的自旋锁:

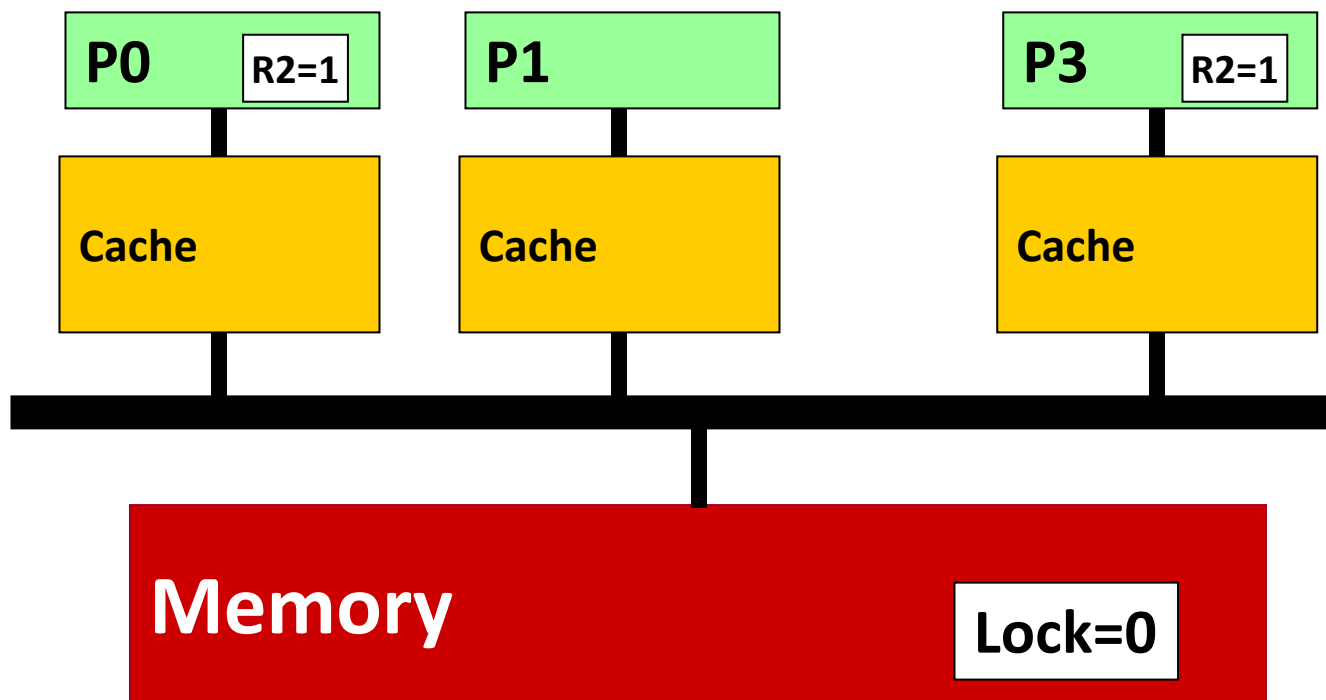
```
lockit:    DADDUI    R2,R0,#1 ;R2 ← 1
           EXCH      R2,0(R1) ;atomic exchange
           BNEZ      R2,lockit ;already locked?
```

- 也称为 *atomic read-modify-write*

如果锁变量0(R1) 在cache中会发生什么?

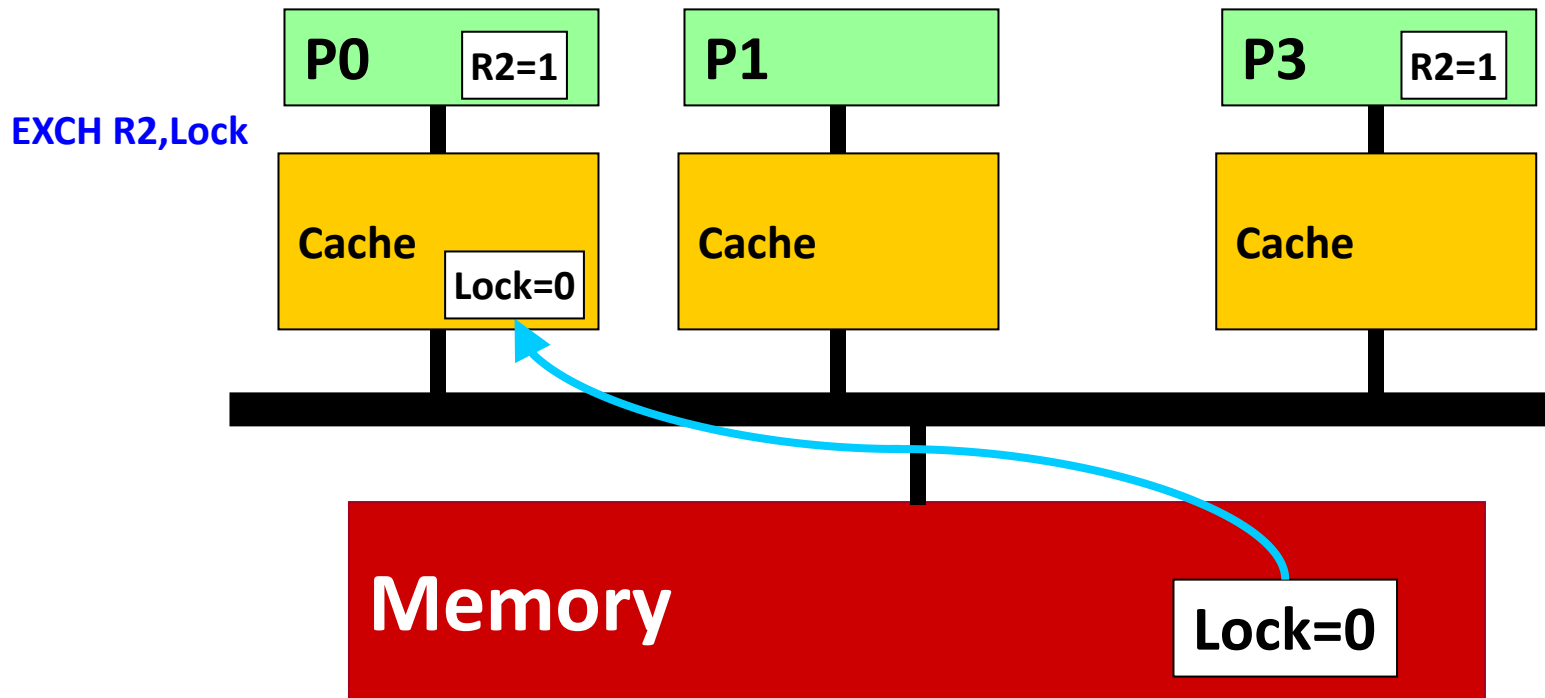
锁变量允许被Cache

- 如果read-modify-write 不是原子的，并且锁变量允许被cache → **race condition**



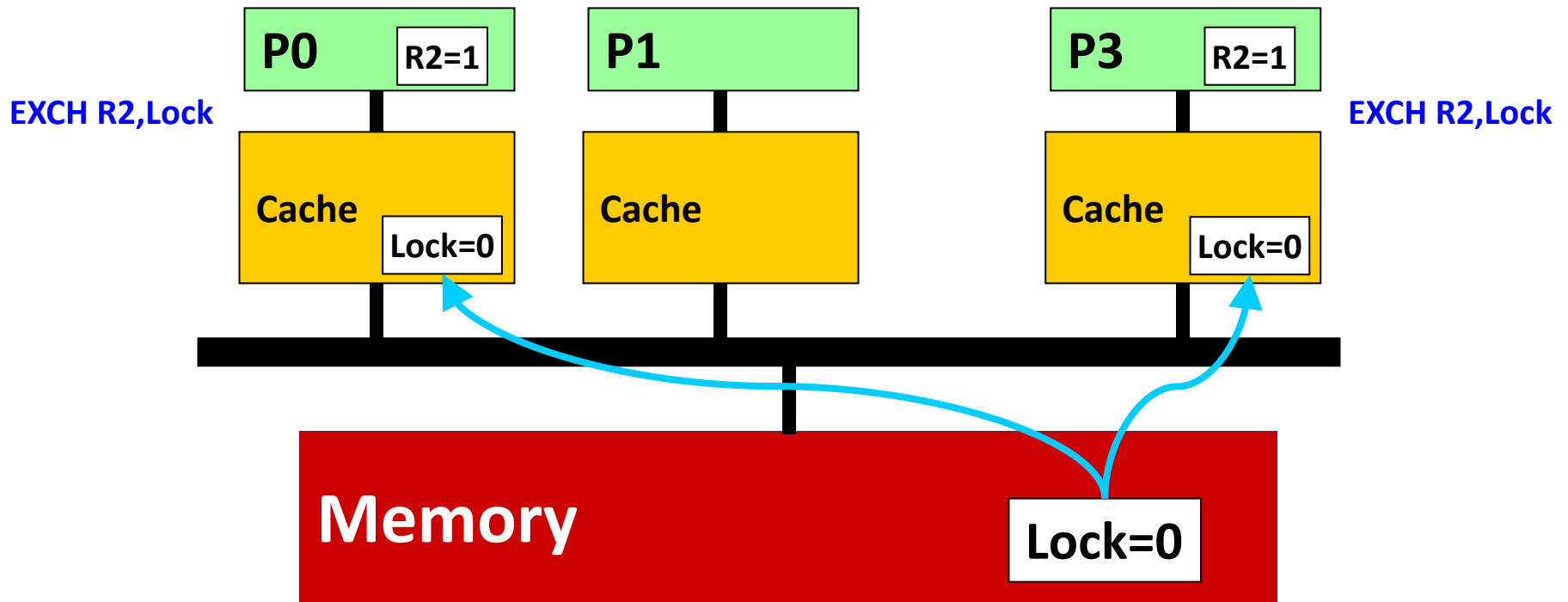
锁变量允许被Cache

- 如果read-modify-write 不是原子的，并且锁变量允许被cache → **race condition**



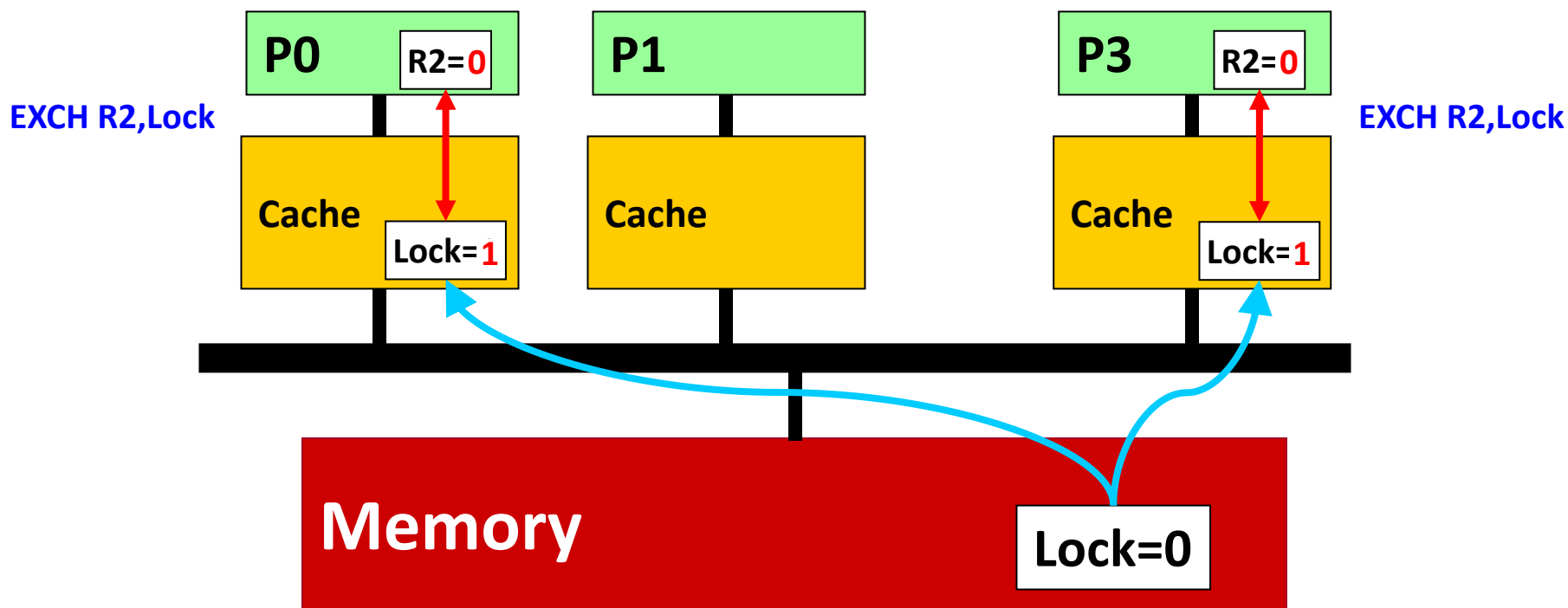
锁变量允许被Cache

- 如果read-modify-write 不是原子的，并且锁变量允许被cache → **race condition**



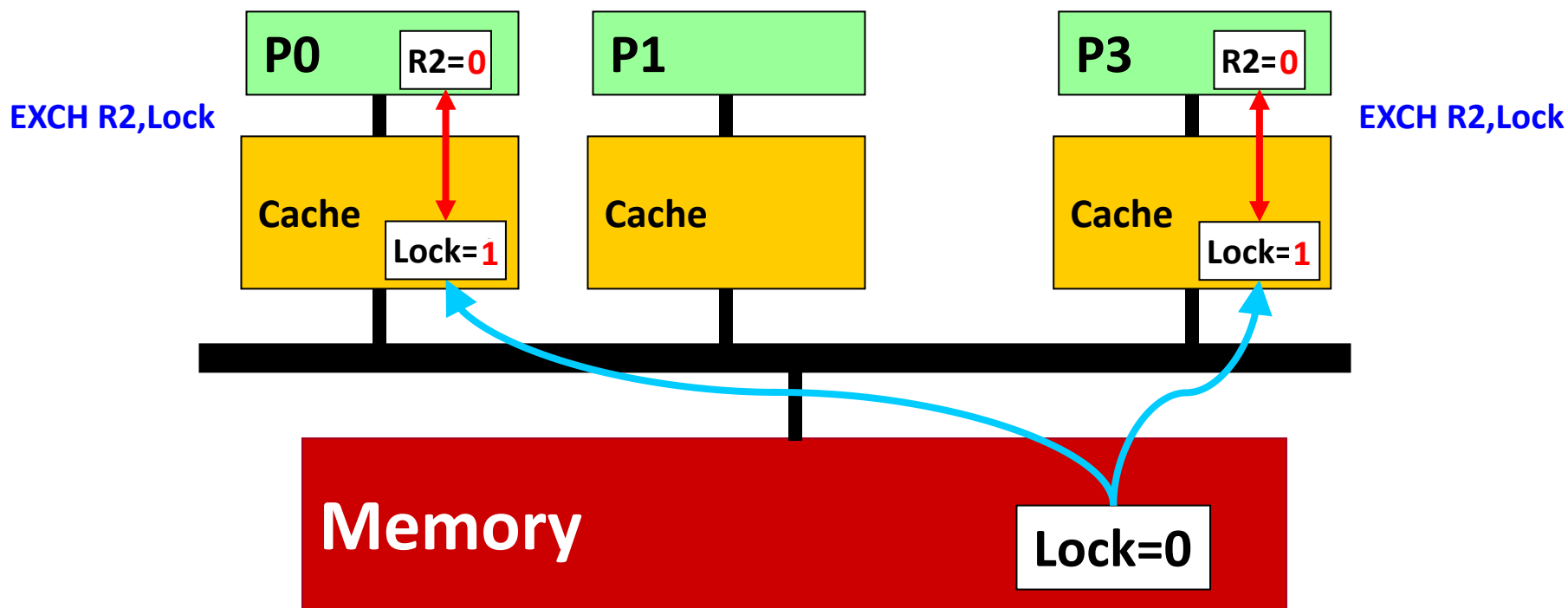
锁变量允许被Cache

- 如果read-modify-write 不是原子的，并且锁变量允许被cache → **race condition**



锁变量允许被Cache

- ❑ 如果read-modify-write 不是原子的，并且锁变量允许被cache → **race condition**
- ❑ 如何保证read-modify-write 是原子的？



基于自旋锁的原子交换

□ **Option 1: 让锁变量 (O(R1)) 一直留在内存，不能被cache**

- 如何保证原子交换 (load-and-store)?
- 总线通信开销很大

□ **Option 2: 允许锁变量被cache**

- 每次交换将尝试写操作
- 很多invalidate信息，锁变量不断更换ownership → 同样很多通信开销

□ **希望可以在cache上实现原子性，同时避免通信开销**

基于自旋锁的原子交换

□ Option 3:

- 处理器在锁变量的本地copy上进行自旋
- 先通过load操作读锁变量
 - 如果是1，自旋
 - 如果是0，尝试原子交换抢锁
- 只有在原子交换抢锁的时候才会有总线通信开销

```
lockit:    LD      R2,0(R1)    ;load lock
           BNEZ      R2,lockit ;not avail.-spin
           DADDUI R2,R0,#1    ;R2 ← 1
           EXCH      R2,0(R1)  ;swap
           BNEZ      R2,lockit ;spin if not 0
```

原子交换过程中保持Cache一致性

□ 每个处理器持续读锁变量

- 读取的是local copy,不会产生coherence通信
- 每个处理器在locally copy上自旋

□ 当锁被释放，值更新为 0

- 写操作将会invalidate所有shared copy
- 每个自旋的处理器再次读锁变量时，发生read miss, 获取新的值 0, 看到0之后, 尝试原子交换抢锁
- 只有一个处理器可以抢到锁，其本地copy变为Modified状态，其他处理器会继续自旋

Load-Linked and Store-Conditional

□ atomic read-modify-write 的问题:

- 一个原子操作包含两次内存操作，不可中断

□ 替代方案: 让一对指令看起来像原子的

- 避免连续的不可中断的内存读和写

□ *Load-locked (LL) and store-conditional (SC)*

- **LL R1,x:** 将内存地址x的值读进R1, 并将地址x保存进link register
- **SC x,R1,R2:** 尝试将R1的值写入地址x, 当LL R1, x之后没有其他指令修改地址x的值时写入成功, 否则失败
 - 写入成功, 返回R2=1; 否则返回R2=0

Load-Linked and Store-Conditional

□ 基于LL & SC实现加锁/解锁

```
lockit: LL    R2,0(R1)    ;load linked
        BNEZ   R2,lockit  ;not free=>spin
        DADDUI R2,R0,#1    ;locked value
        SC     0(R1),R2,R3 ;store
        BEQZ   R3,lockit  ;branch if fails
```

Critical Section

```
ST      0(R1),#0
```

LL读取锁的值，如果是0，抢锁，执行SC操作，如果SC成功，R3返回1，进入critical section，如果SC失败，返回0，重新抢锁

Load-Linked and Store-Conditional

□ 基于LL & SC实现原子 (读数->加1->写回) :

```
try: LL    R2,0(R1)    ; load linked
      DADDUI R2,R2,#1    ; increment
      SC     0(R1),R2,R3 ; store
      BEQZ   R3,try      ; branch SC fails
```

□ 需要同步机制处理访问顺序,互斥, 屏障等

□ 同步机制的硬件支持

- 禁用中断
- 原子交换
- Load linked/store conditional instructions